

Ledger based self orchestration cho mô hình ngôn ngữ

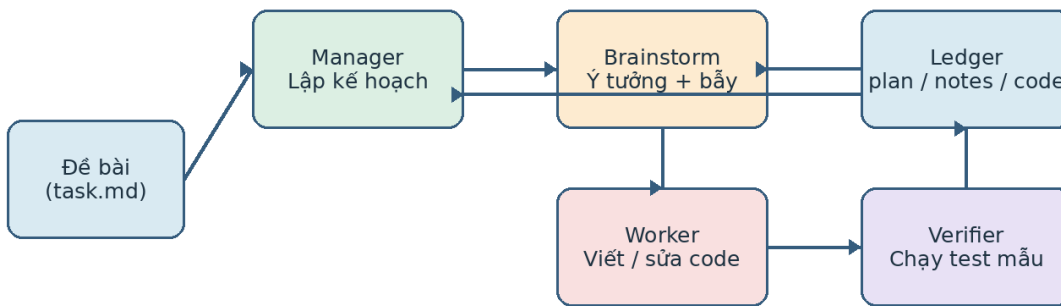
Đọc bài báo **Zero Shot Self Orchestration with Ledger Based Control** theo cách trực giác, có ví dụ và phân tích thực nghiệm

Mục tiêu của tài liệu này là giúp người mới với machine learning hiểu chính xác bài báo đang làm gì, vì sao cách làm có thể tốt hơn một lần gọi đơn lẻ, kết quả nào thực sự thuyết phục, và khi nào nó thất bại. Nguồn chính là PDF đính kèm 19 trang; các nguồn trực tuyến chỉ được dùng để kiểm tra bối cảnh và khả năng tái lập.

Tóm tắt một phút

Bài báo không huấn luyện một mô hình mới. Nó lấy cùng một mô hình, gọi nhiều lần trong các context mới, rồi cho các lần gọi đó làm các vai trò khác nhau: một manager lập kế hoạch, các worker tìm ý tưởng hoặc viết code, và một verifier chạy test. Điểm khác biệt cốt lõi là mọi người cùng đọc và ghi vào một “sổ cái” trên filesystem: kế hoạch, danh sách việc, ghi chú và file code hiện tại.

Kết quả chính trong thí nghiệm pinned-backend: GPT-5.6-Luna tăng từ 70.4% lên 81.2%, GPT-5.6-Terra từ 80.8% lên 88.0%, còn Qwen3.8-27B từ 69.2% lên 92.4% pass@1. Nhưng hệ thống tốn khoảng gấp ba token, và không phải mô hình nào cũng tốt hơn. Thông điệp đúng là “tổ chức suy luận ở thời điểm chạy có thể thay thế một phần việc tăng kích thước mô hình”, chứ không phải “multi-agent luôn thắng”.



Sau mỗi vòng, manager đọc trạng thái đã chốt lọc và chọn đúng một việc tiếp theo

Hình 1. Luồng manager–worker–verifier và vai trò của ledger. Vẽ lại từ Hình 1 và Mục 3 của PDF.

1. Vấn đề mà bài báo muốn giải quyết

Một mô hình mạnh thường được dùng theo kiểu: đưa đề bài vào một lần, chờ nó suy luận rồi lấy code. Cách này có hai điểm yếu. Thứ nhất, mô hình có thể lao vào viết code trước khi tìm ra phép rút gọn của bài toán. Thứ hai, một câu trả lời rất dài có thể bị cắt ở giới hạn token hoặc bị mắc kẹt trong việc tự kiểm tra vô tận.

Nếu một hệ multi-agent tốt hơn, ta cần hỏi: nó tốt hơn vì có nhiều “bộ não” độc lập, vì có thêm token, vì prompt khác, hay vì có bộ nhớ ngoài? Bài báo cố định mô hình và bộ dữ liệu rồi thêm một scaffold cụ thể, nhằm cô lập phần lợi ích đến từ tổ chức và bộ nhớ bền vững.

2. Ba ý tưởng trung tâm

2.1. Zero-shot self orchestration nghĩa là gì?

Zero-shot ở đây không có nghĩa mô hình chưa từng học coding. Nó nghĩa là manager không được huấn luyện riêng để điều phối và không được cho ví dụ đặc thù của benchmark. Tất cả vai trò vẫn là cùng một model, chỉ khác lời nhắc và context.

Một phép so sánh đời thường: thay vì nhờ một người giải bài trong một buổi nói chuyện dài, ta cho cùng người đó lần lượt đóng vai người lập kế hoạch, người tìm hướng giải, người lập trình và người kiểm tra. Chất lượng có thể tăng vì mỗi lần trở lại, người đó nhìn thấy một bản tóm tắt sạch hơn.

2.2. Ledger là bộ nhớ ngoài có cấu trúc

Tệp	Nội dung	Tác dụng
task.md	Đề bài gốc	Không để mất yêu cầu
plan.md	Chiến lược 3–6 câu	Giữ phép rút gọn / mục tiêu lớn
tasks.json	Danh sách việc còn lại	Biến suy nghĩ thành queue có trạng thái
notes.md	Ý tưởng, bẫy, chứng minh, kết quả	Truyền phần quan trọng sang context mới
solution.py	Code tốt nhất hiện tại	Artifact để worker kế tiếp sửa tiếp

Điểm tinh tế là worker không nhận toàn bộ transcript cũ. Nó nhận plan, notes, solution và đúng một task. Nhờ vậy, context mới chứa thông tin đã được chọn lọc thay vì hàng trăm đoạn tự thoại, nhưng trạng thái quan trọng vẫn sống qua nhiều lượt.

2.3. Manager không chạy pipeline cố định

Một pipeline cố định sẽ luôn làm A rồi B rồi C. Ở đây manager đọc tình hình sau mỗi vòng: gộp task trùng, đánh dấu task xong, thêm task mới nếu cần, và chọn đúng một việc tiếp theo. Nếu test mẫu fail, tín hiệu fail có quyền phủ quyết việc manager vừa nghĩ là đã xong.

3. Cách hệ thống chạy một bài toán

Bước	Điều xảy ra
Bước 1 — Plan	Manager đọc đề, ghi chiến lược và tạo 3–6 task mới.
Bước 2 — Brainstorm	Worker đầu tiên chỉ tìm khó khăn, hướng giải, bẫy; chưa viết lời giải hoàn chỉnh.
Bước 3 — Manage	Manager đọc ledger, làm sạch queue và chọn task quan trọng nhất.
Bước 4 — Do the task	Worker mới thực hiện task, cập nhật solution.py và notes.md.
Bước 5 — Verify	Nếu có code mới, chạy public stdin tests; fail thì buộc vòng lặp tiếp tục.
Bước 6 — Finalize	Khi dừng do hết vòng, lập task, hoặc queue rỗng mà chưa có sign-off sạch, gọi worker hoàn thiện.

Các guard quan trọng: tối đa 10 vòng manager–worker; nếu manager giao lại đúng task vừa giao thì dừng vì không tiến bộ; nếu worker chạm token cap thì gọi một summarizer ngắn để cứu ý tưởng dang dở. plan.md bị giới hạn 4,000 ký tự và notes.md 8,000 ký tự.

Ví dụ nhỏ: bài toán đường đi trên cây

Giả sử đề yêu cầu tìm độ dài lớn nhất của một đoạn đường trên cây mà không lặp giá trị. Một lần gọi đơn lẻ có thể giữ một tập độ dài cho từng cây con, rồi bị phình trạng thái. Trong transcript minh họa, brainstorm ghi rõ phép quy về “longest substring without repeating characters” dọc từng đường từ gốc đến lá; worker sau đó triển khai cửa sổ trượt bằng last-seen depth. Điều quan trọng không phải manager tự tạo ra một model mới, mà là phép rút gọn được ghi lại trước khi code.

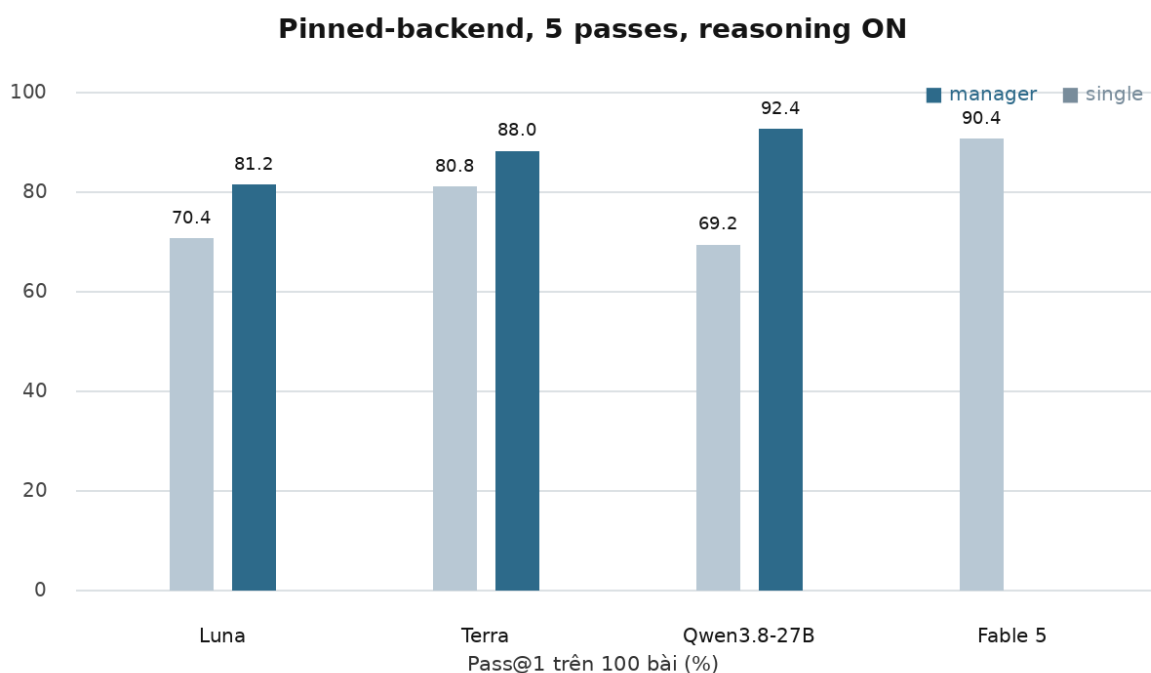
Một ví dụ khác: nếu cách ngây thơ có độ phức tạp $O(N \cdot M)$ xấp xỉ 4×10^{10} , brainstorm có thể cảnh báo phải dùng range query trước. Cảnh báo này làm thay đổi hướng giải ngay từ đầu thay vì đợi code chạy quá chậm.

4. Thí nghiệm được thiết kế như thế nào?

Benchmark là 100 bài khó mới nhất trong LiveCodeBench release_v6, chọn theo ngày contest để giảm nguy cơ nhiễm dữ liệu huấn luyện. LiveCodeBench là benchmark liên tục cập nhật, lấy bài từ LeetCode, AtCoder và Codeforces, đồng thời có các kịch bản code generation, code execution và test output prediction [2]. Bài báo chỉ báo cáo code generation trên tập LCB-100.

Pass@1 trong tài liệu này được hiểu đơn giản: trong 100 bài, bao nhiêu bài có chương trình cuối cùng vượt qua hidden tests. Điều kiện headline chạy 5 passes độc lập, cap 128k output tokens và reasoning bật. So sánh chính dùng cùng model, cùng đề, cùng temperature; khác nhau ở single call và manager scaffold.

Model	Single call	Manager	Mức tăng
GPT-5.6-Luna	70.4%	81.2%	+10.8 điểm
GPT-5.6-Terra	80.8%	88.0%	+7.2 điểm
Qwen3.8-27B	69.2%	92.4%	+23.2 điểm
Claude Fable 5	90.4%	—	Không chạy manager



Hình 2. Kết quả headline theo số liệu trong PDF đính kèm, Hình 2.

Ba cải thiện trong pinned-backend đều có ý nghĩa theo paired sign-flip permutation test ở mức $p < 10^{-4}$ trong PDF. Phân tích McNemar cũng cho thấy manager-only wins áp đảo single-only wins: lần lượt 47/11 cho Terra, 72/18 cho Luna và 124/8 cho Qwen trên 500 problem-passes mỗi model.

5. Vì sao kết quả có thể tốt?

5.1. Decomposition: tách phép nghĩ khỏi phép viết

Các bài coding khó thường gồm nhiều lớp: tìm invariant, chứng minh độ phức tạp, chọn cấu trúc dữ liệu, xử lý input/output và viết code không lỗi. Khi một call phải làm tất cả, lỗi ở lớp đầu có thể kéo theo code sai. Brainstorm buộc “ý tưởng cốt lõi” xuất hiện dưới dạng ghi chú trước khi worker triển khai.

5.2. Persistent context: giữ insight, bỏ lan man

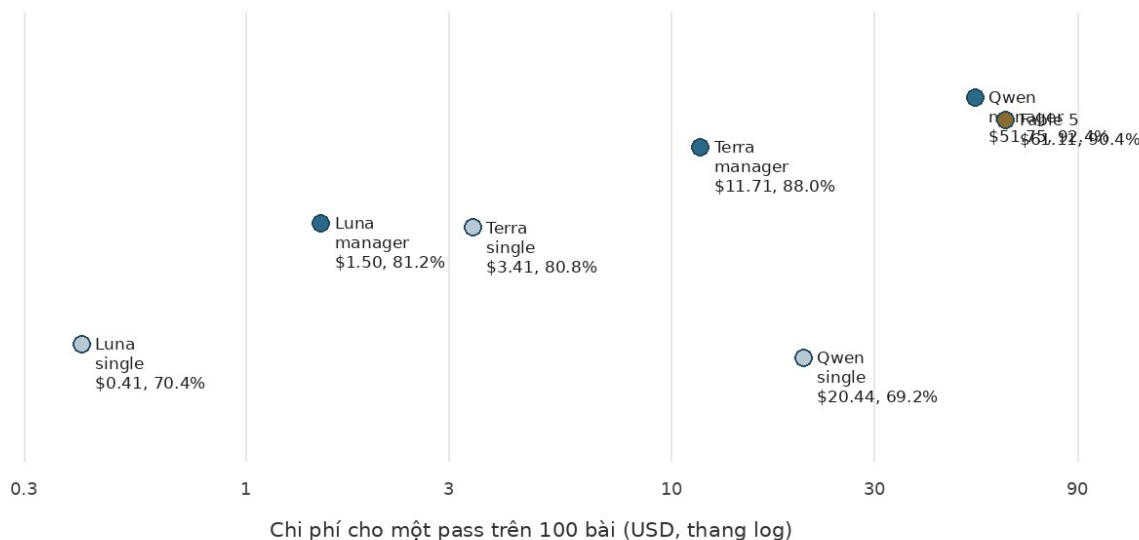
Nếu model đã tìm ra phép quy về min-cut nhưng tiếp tục tự hỏi đi hỏi lại hàng nghìn edge case, insight có thể nằm ở giữa một transcript bị cắt. Ledger đặt insight đó vào plan.md hoặc notes.md, nên worker sau đọc thấy ngay. Đây là bộ nhớ ngoài, không bị buộc phải nằm trong một response duy nhất.

5.3. Bounded calls: nhiều cơ hội hữu ích hơn một lần độc thoại

Trong Qwen3.8-27B, single arm có 150/500 lần chạm cap và 35/500 problem-passes không sinh code; manager chỉ còn 5 cap hits và 0 no-code. Manager có thể biến một worker bị cắt thành “mất một vòng”, thay vì mất toàn bộ bài. PDF ước tính việc cứu các empty cells đóng góp khoảng 5.6 điểm, tức khoảng một phần tư mức tăng của Qwen; phần còn lại đến từ giải bài thực sự tốt hơn.

6. Chi phí: tổ chức tốt hơn không miễn phí

Chi phí - độ chính xác



Hình 3. Chi phí tăng cùng độ chính xác; mỗi tên nối single call với manager của cùng model.

Một pass gồm cả 100 bài. Luna tăng từ \$0.41 lên \$1.50; Terra từ \$3.41 lên \$11.71; Qwen từ \$20.44 lên \$51.75; Fable 5 single-only là \$61.11. Vì vậy Terra có manager đạt 88.0% gần Fable 5 90.4% với khoảng 19% chi phí. Qwen manager đạt 92.4%, nhỉnh hơn Fable 5 về điểm nhưng chênh lệch nằm trong sai số; ưu thế thực tế của Qwen còn phụ thuộc chi phí host model.

Cần hiểu đúng “rẻ hơn”: đây không phải so sánh công bằng theo cùng số token. Manager dùng nhiều token hơn single call. Câu hỏi của bài báo là: thêm tiền cho scaffold trên model rẻ có mua được accuracy rẻ hơn so với chuyển sang model lớn/đắt hay không? Với các cặp được báo cáo, câu trả lời là có.

7. Khi hệ thống thất bại?

Lợi ích không phổ quát. Trong các điều kiện OpenRouter reasoning-off, Qwen3.6-35B giảm từ 35% xuống 27% ở cap 128k, và gần như không cải thiện ở 16k. Một transcript cho thấy ideation đã tìm đúng optimization mà single call dùng, nhưng sau đó tự thuyết phục rằng nó quá rủi ro; worker triển khai phương án chậm hơn và có bug. Ledger khi đó trở thành cơ chế “đóng băng sai lầm”.

Có ba rủi ro khái quát: (1) anchoring — mọi worker kế tiếp kế thừa framing sai; (2) trust without verification — sample tests không bao phủ hidden cases; (3) extra compute — manager tiêu token và có thể làm nhiễu một lời giải vốn đã đúng. Cách cải thiện hợp lý do tác giả gợi ý là thêm fresh-perspective workers không đọc notes cũ, và chạy generated test suite trước khi chấp nhận candidate.

8. Phụ lục evaluator và giới hạn bằng chứng

Tác giả phát hiện lỗi trong evaluator LiveCodeBench: binary readline() của mock stdin luôn trả lại dòng đầu, khiến chương trình dùng sys.stdin.buffer.readline() bị chấm sai. Họ sửa mock bằng BytesIO rồi chấm lại toàn bộ generation; đồng thời sửa cách chấm output có nhiều đáp án đúng, floating-point tolerance và sys.stdout.buffer.write(). Sau sửa, mọi số liệu trong Mục 5 của PDF đều là số đã re-score.

Điều này làm bài báo đáng tin hơn ở điểm minh bạch, nhưng cũng cho thấy score phụ thuộc evaluator. Một giới hạn khác là năm model OpenRouter chịu ảnh hưởng routing/provider health và phần lớn chỉ có one-pass; các headline claims nên đọc từ pinned-backend set. Table 5 không có manager arm, Qwen single 128k là replay từ generation 250k, và tất cả kết quả đều chỉ trên một họ benchmark coding.

Về phiên bản nguồn: PDF đính kèm có tiêu đề phụ “Improves Coding in Language Models” và headline 88.0/90.4/92.4 như trên. Bản arXiv hiện được lập chỉ mục trực tuyến có abstract với một số con số khác, cho thấy manuscript đã được cập nhật. Tài liệu này cố ý dùng PDF đính kèm làm nguồn chuẩn và không trộn số liệu giữa các phiên bản [1].

9. Kết luận dễ nhớ

Bài báo đang thử một giả thuyết rất cụ thể: cùng một model có thể giải code tốt hơn nếu nó được tổ chức thành nhiều lần gọi ngắn, có manager chọn việc, có ledger giữ trạng thái và có verifier kéo hệ thống quay lại khi test fail. Kết quả mạnh nhất xuất hiện khi single call bị nghiền ở khâu tổ chức suy nghĩ, context hoặc token cap; kết quả yếu hoặc âm xuất hiện khi model đã có hướng đúng nhưng scaffold làm nó đổi hướng.

Câu chốt: ledger không tạo thêm kiến thức thần kỳ. Nó làm cho kiến thức/insight mà model đã có được ghi đúng chỗ, được chia thành việc vừa sức, và còn tồn tại sau khi một context kết thúc. Đổi lại, ta trả bằng token, độ trễ và nguy cơ lan truyền một kế hoạch sai.

10. Nếu muốn áp dụng ý tưởng này

- Giữ baseline single call trên đúng model và đúng tập bài; đừng chỉ đo manager một mình.
- Bắt đầu với ledger nhỏ: problem, plan, tasks, notes, current solution. Giới hạn kích thước notes để tránh biến ledger thành transcript mới.
- Tách brainstorm khỏi implementation ở các bài có phép quy khó; nhưng cho phép fresh-perspective worker khi rủi ro anchoring cao.
- Chạy public tests và generated tests trước khi cho một worker ghi đề solution tốt.
- Đo đồng thời accuracy, cost, latency, truncation, empty outputs và số lần manager-only win/loss.

Nguồn

[1] Victor Gao, Vida Khosrowshahi, Ali Khosrowshahi, Xihao Sun, Juhyun Lee, Simon (Sang Won) Lee. “Zero-Shot Self-Orchestration with Ledger-Based Control Improves Coding in Language Models.” PDF đính kèm: iclr2027_conference.pdf, 19 trang, under review at ICLR 2027. Các số liệu và mô tả phương pháp trong tài liệu này lấy từ nguồn này, đặc biệt Mục 3–6 và Phụ lục A–D.

[2] Naman Jain et al. “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.” arXiv:2403.07974, 2024. <https://arxiv.org/abs/2403.07974>. Kho chính thức: <https://github.com/LiveCodeBench/LiveCodeBench>. Dùng để giải thích benchmark và release_v6.

[3] Kho mã và run data được bài báo liên kết: <https://github.com/slee-persis/GVS5H>. README mô tả engine single/manager, cách chạy release_v6 và vị trí workspace transcripts. Đây là nguồn tái lập, không phải nguồn thay thế cho PDF đính kèm.